
Bases de Datos 2

Procesamiento y Optimización de consultas
Gestión de índices y vistas

¿Qué vamos a aprender hoy?

- El "Qué": SQL como lenguaje declarativo.
- El Viaje de la Consulta: Análisis, Optimización y Ejecución.
- El Cerebro: Optimizador Heurístico vs. Basado en Costos.
- Las Herramientas de Poder: Índices y Vistas Materializadas.
- De Junior a Senior: El Plan de Ejecución como brújula.

La naturaleza declarativa de SQL

A diferencia de los lenguajes de programación imperativos (como C o Java) donde debemos determinar cada paso lógico, SQL es un lenguaje declarativo de alto nivel. Esto significa que el usuario especifica qué datos desea obtener o manipular, pero no cómo deben realizarse físicamente esas operaciones.

- La analogía del taxi/uber: Cuando piden un taxi, solo indican el destino (el "qué"). No le dicen al conductor qué calles tomar, en qué cambio poner la marcha, cuándo acelerar o cuándo frenar (el "cómo"). El motor de la base de datos es ese "conductor experto" que decide la ruta más rápida basándose en el tráfico (índices) y la distancia (volumen de datos)

¿Cómo piensa un programador vs. Cómo piensa SQL?

- Enfoque Imperativo (Java/Python): El programador define el bucle y el orden.

```
resultado = []
for c in clientes:
    if c.provincia == 'BSAS':
        for p in pedidos:
            if p.id_cliente == c.id:
                resultado.append(...)
```

- Enfoque Declarativo (SQL): El programador define el resultado. El motor decide el bucle.

```
SELECT nombre FROM clientes
JOIN pedidos ON ...
WHERE provincia = 'BSAS';
```

Consulta de referencia para la presentación

```
SELECT c.nombre, SUM(p.importe) AS total  
FROM clientes C  
JOIN pedidos P ON C.id_cliente = P.id_cliente  
WHERE P.fecha >= '2026-01-01' AND C.provincia = 'Buenos Aires'  
GROUP BY C.nombre;
```

Durante toda la clase vamos a volver a esta consulta.

Supuestos didácticos:

- Tablas medianas/grandes
- pedidos tiene muchos registros

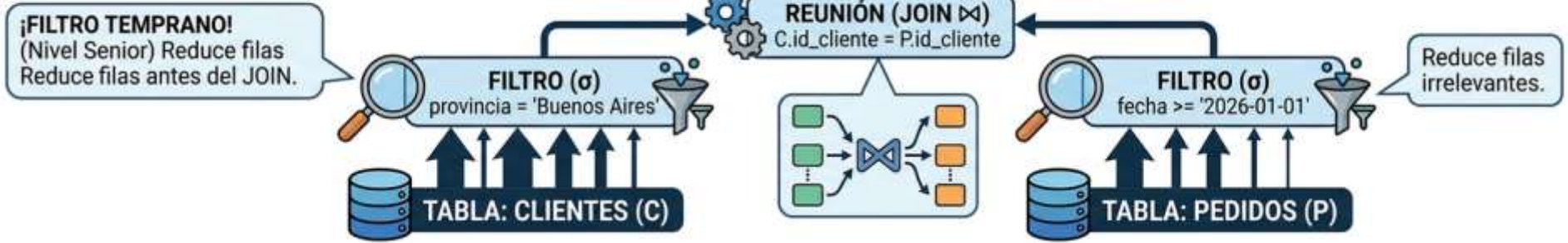
Ciclo de vida y Fases del Procesamiento

El procesamiento de una consulta es una secuencia de actividades para extraer datos de la base de datos. El flujo sigue este "pipeline":

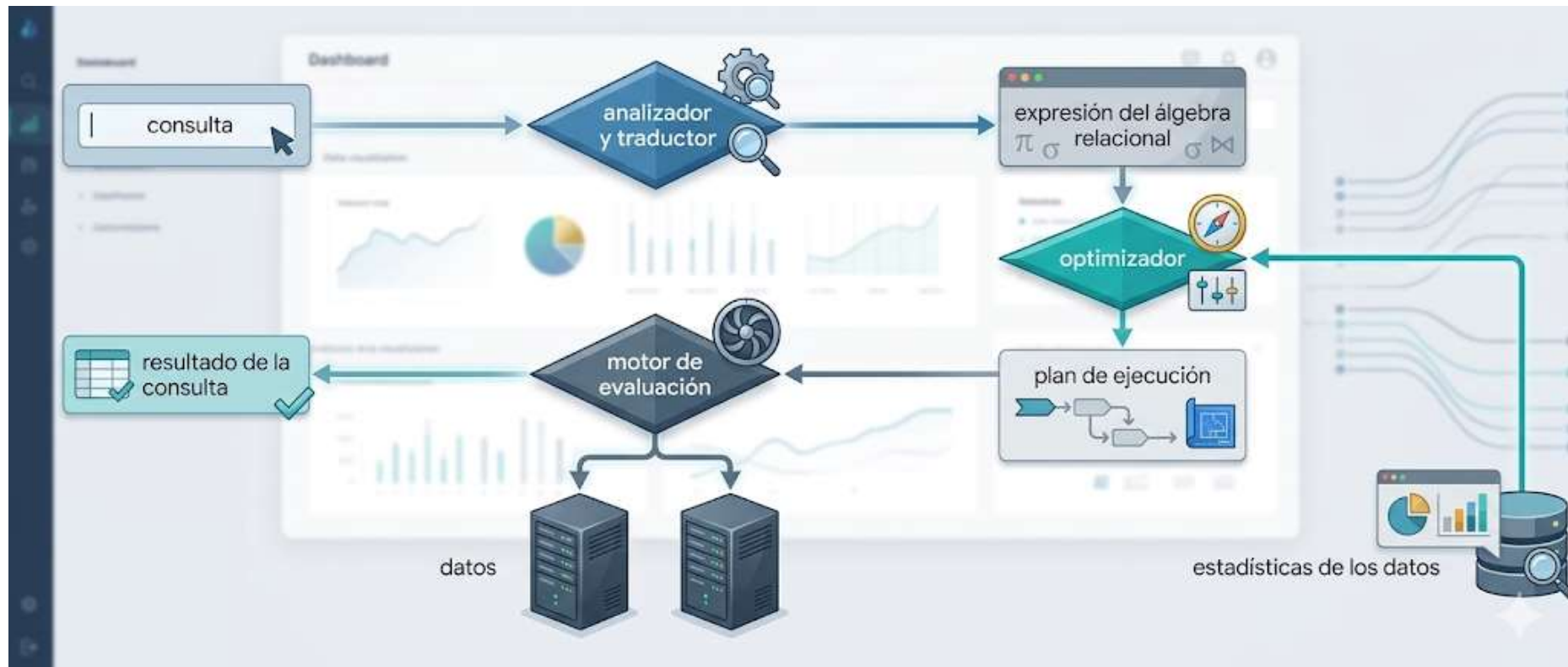
- 1. Análisis y Traducción:** El SGBD recibe el SQL y lo traduce a una representación interna utilizable, generalmente basada en el álgebra relacional extendida.
- 2. Optimización:** Es el "cerebro" del proceso. El sistema transforma la consulta original en un plan de ejecución equivalente que minimice el costo.
- 3. Evaluación (Ejecución):** El motor de ejecución recibe el plan, accede a los datos en el disco y devuelve los resultados.

PLAN DE EJECUCIÓN LÓGICO DE LA CONSULTA SQL (AYUDA MEMORIA)

```
SELECT c.nombre, SUM(p.importe) AS total
FROM clientes C
JOIN pedidos P ON C.id_cliente = P.id_cliente
WHERE P.fecha >= '2026-01-01' AND C.provincia = 'Buenos Aires'
GROUP BY C.nombre;
```



Ciclo de vida y Fases del Procesamiento



Ciclo de vida y Fases del Procesamiento



1. Análisis

- Verificación de sintaxis (gramática)
- Verificación semántica (catálogo)
- **Salida: Álgebra Relacional**



2. Optimización

- Lógica: reescribir el álgebra
- Física: elegir índices y algoritmos
- **Salida: Plan de ejecución**



3. Evaluación (Ejecución)

- Procesamiento del plan
- Acceso a disco y memoria
- **Retorno de datos**

De SQL al Álgebra Relacional - no entra en el parcial

Una consulta SQL se traduce, en primer lugar, a una expresión extendida equivalente de álgebra relacional (representada como una estructura de datos de árbol de consultas) que es optimizada posteriormente.

Por regla general, las consultas SQL se descomponen en bloques de consulta que forman las unidades básicas que se pueden traducir a los operadores algebraicos y después ser optimizadas.

Un bloque de consulta contiene una única expresión SELECT-FROM-WHERE, y también cláusulas GROUP BY y HAVING en el caso de que éstas formen parte del bloque. Por tanto, las consultas anidadas dentro de una consulta se identifican como bloques de consulta independientes.

De SQL al Álgebra Relacional - no entra en el parcial

Cada bloque SQL puede traducirse a diversas expresiones del álgebra relacional.

```
SELECT sueldo  
FROM profesores  
WHERE sueldo < 75000;
```

→ $\sigma_{\text{sueldo} < 75000} (\pi_{\text{sueldo}} (\text{profesor}))$

→ $\pi_{\text{sueldo}} (\sigma_{\text{sueldo} < 75000} (\text{profesor}))$

Los **diferentes planes de evaluación** de una consulta dada **pueden tener costos distintos**.

Es responsabilidad del sistema construir un plan de evaluación de la consulta que minimice el costo de la ejecución; esta tarea se denomina optimización de consultas.

El costo de una consulta

El coste de la evaluación de una consulta se puede expresar en términos de **diferentes recursos**, incluyendo los **accesos a disco**, el tiempo que tarda la **CPU** en ejecutar una consulta y, en sistemas de bases de datos distribuidos o paralelos, el coste de la **comunicación**.

En grandes sistemas de bases de datos, el coste de acceso a los datos en disco es normalmente el más importante, ya que los accesos a disco son lentos comparados con las operaciones en memoria. Además, la velocidad de la CPU ha aumentado mucho más rápidamente que las velocidades de los discos. Por tanto, lo más probable es que el tiempo empleado en operaciones del disco siga dominando en el tiempo total de ejecución de una consulta.

Tiempo de respuesta y costo

El tiempo de respuesta para un plan de evaluación de una consulta (es decir, el tiempo de reloj para ejecutar el plan), suponiendo que no se está realizando ninguna otra actividad en la computadora, tendría en cuenta todos los costes y se podría usar como una medida del coste del plan. Desafortunadamente, **el tiempo de respuesta de un plan resulta muy difícil de estimar sin ejecutar realmente el plan**, porque:

1. El tiempo de respuesta depende del contenido de la memoria intermedia cuando comienza la ejecución de la consulta; no se puede conocer esta información cuando se optimiza la consulta y resulta difícil tenerlo en cuenta aunque estuviese disponible.
2. En un sistema con varios discos, el tiempo de respuesta depende de cómo se distribuyan los accesos entre los discos, lo que resulta muy difícil de estimar sin una información detallada de la distribución de los datos.

En lugar de intentar minimizar el tiempo de respuesta, los optimizadores suelen intentar **minimizar el consumo de recursos total** de un plan de consulta.

Fase 1: Análisis (parser) y validación

Antes de intentar ejecutar nada, el Analizador (Parser) realiza dos controles críticos:

- Análisis Sintáctico: Comprueba que la sentencia cumpla las reglas gramaticales del SQL (ej. que no falte un FROM o que las comas estén en su sitio).
- Análisis Semántico (Validación): El motor consulta el catálogo del sistema (metadatos) para verificar que las tablas y columnas existan, que los tipos de datos sean compatibles y que el usuario tenga los permisos necesarios para acceder a esa información



Representación interna y Álgebra Relacional

Una vez validada, la consulta se convierte en un **Árbol de Consulta** (Query Tree). Los nodos de este árbol son operaciones del álgebra relacional, que es el lenguaje formal interno del motor.

- Selección (σ): Funciona como un filtro horizontal de filas.
- Proyección (π): Realiza un corte vertical, eligiendo sólo las columnas necesarias.
- Producto Cartesiano ($\times$): Combina todas las filas de una tabla con todas las de otra.
- Join ($\bowtie$): Combina tablas basándose en una condición de emparejamiento (generalmente claves primarias y foráneas)

Repaso - Orden de evaluación de las consultas

El orden lógico de evaluación de una consulta SQL no es el mismo que el orden en que la escribimos. El motor sigue una secuencia definida:

1. **FROM** (incluye los **JOINS**) → se arman las combinaciones de filas.
2. **WHERE** → se filtran las filas individuales (antes de agrupar).
3. **GROUP BY** → se forman los grupos con las filas que sobrevivieron al WHERE.
4. **HAVING** → se filtran los grupos resultantes.
5. **SELECT** → se proyectan las columnas o agregados.
6. **DISTINCT** — si aplica, elimina duplicados del resultado proyectado.
7. **ORDER BY** → se ordena el resultado final.
8. **LIMIT** → se recorta la cantidad de filas devueltas.

Árbol canónico de la consulta

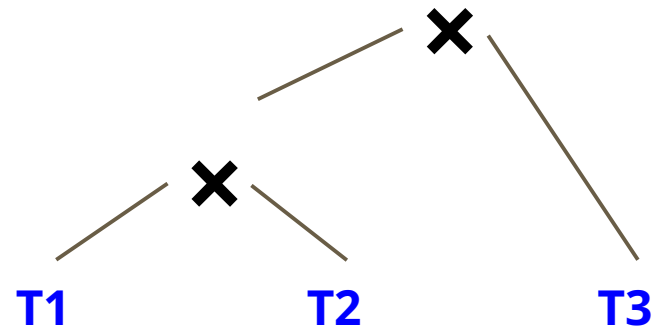
SELECT col1, col2, col3

FROM T1, T2, T3

WHERE T1.a = T2.a AND T2.b = T3.b

$\pi_{col1, col2, col3}$

$\sigma_{T1.a = T2.a \text{ AND } T2.b = T3.b}$



Consulta de referencia para la presentación

```
SELECT c.nombre, SUM(p.importe) AS total  
FROM clientes C  
JOIN pedidos P ON C.id_cliente = P.id_cliente  
WHERE P.fecha >= '2026-01-01' AND C.provincia = 'Buenos Aires'  
GROUP BY C.nombre;
```

Durante toda la clase vamos a volver a esta consulta.

Supuestos didácticos:

- Tablas medianas/grandes
- pedidos tiene muchos registros

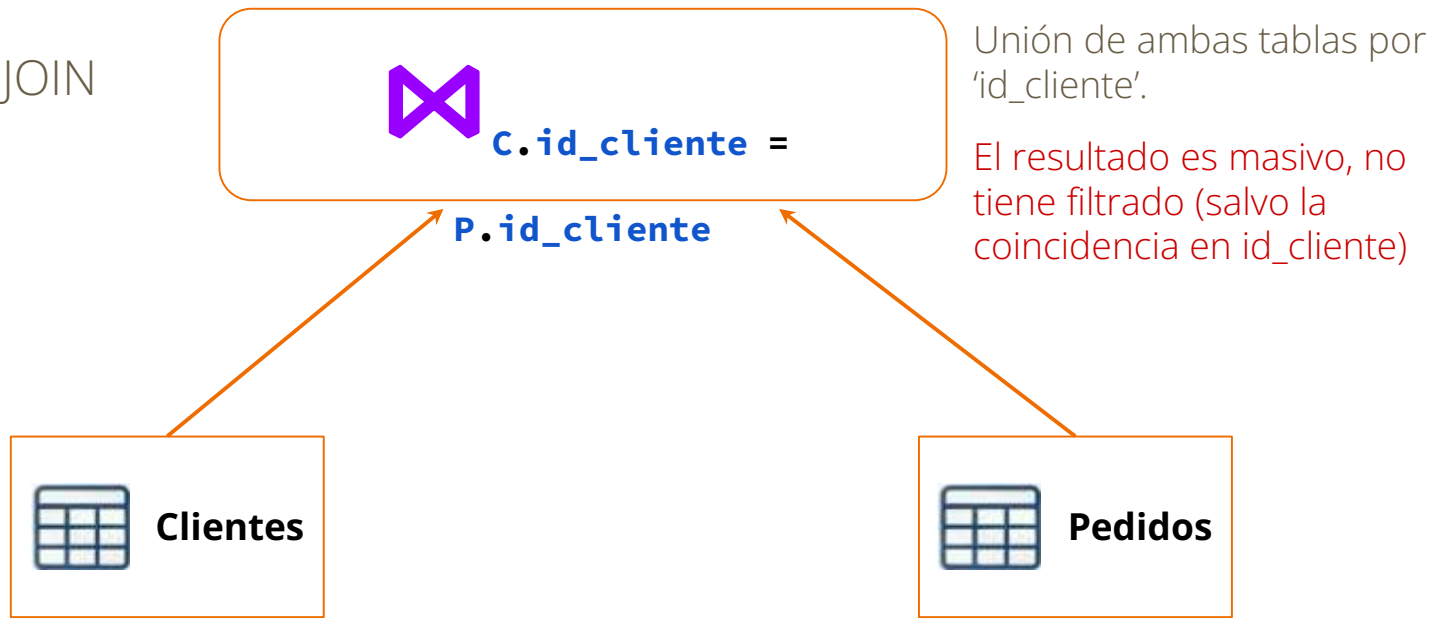
Árbol de la consulta de referencia - Nivel 1



Nivel 1: las hojas (fuentes de datos)

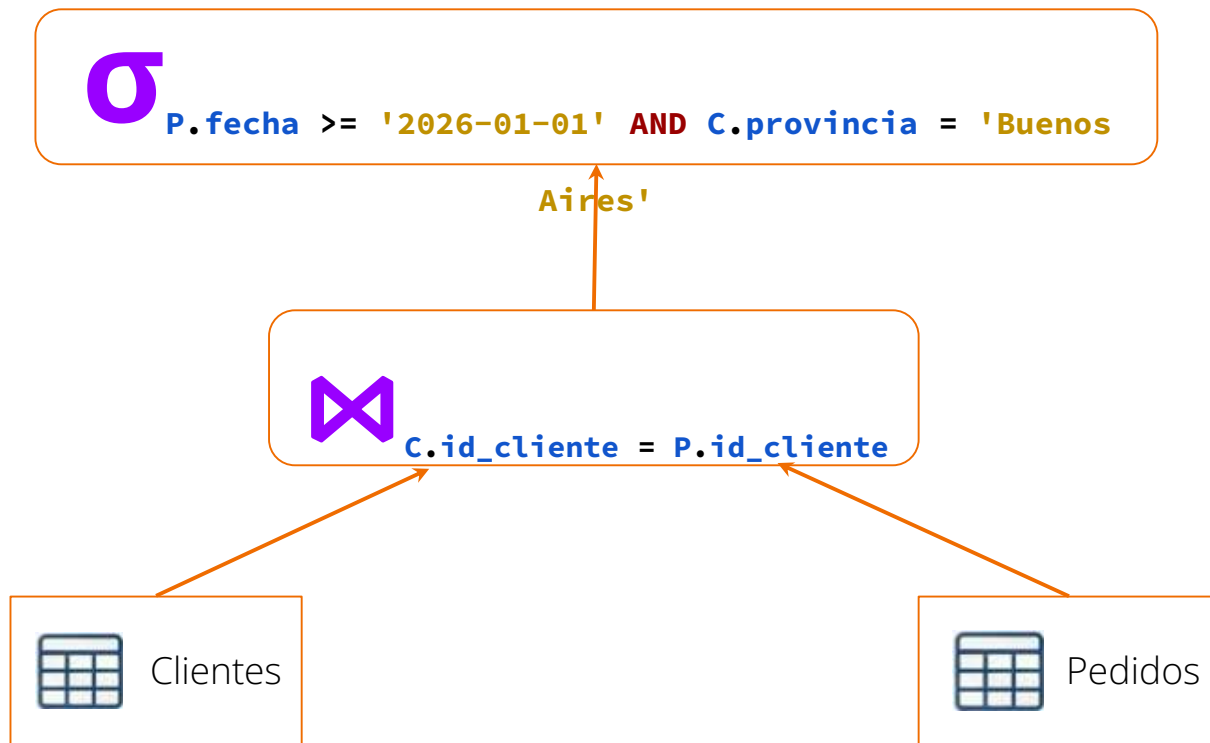
Árbol de la consulta de referencia - Nivel 2

Nivel 2: el nodo JOIN



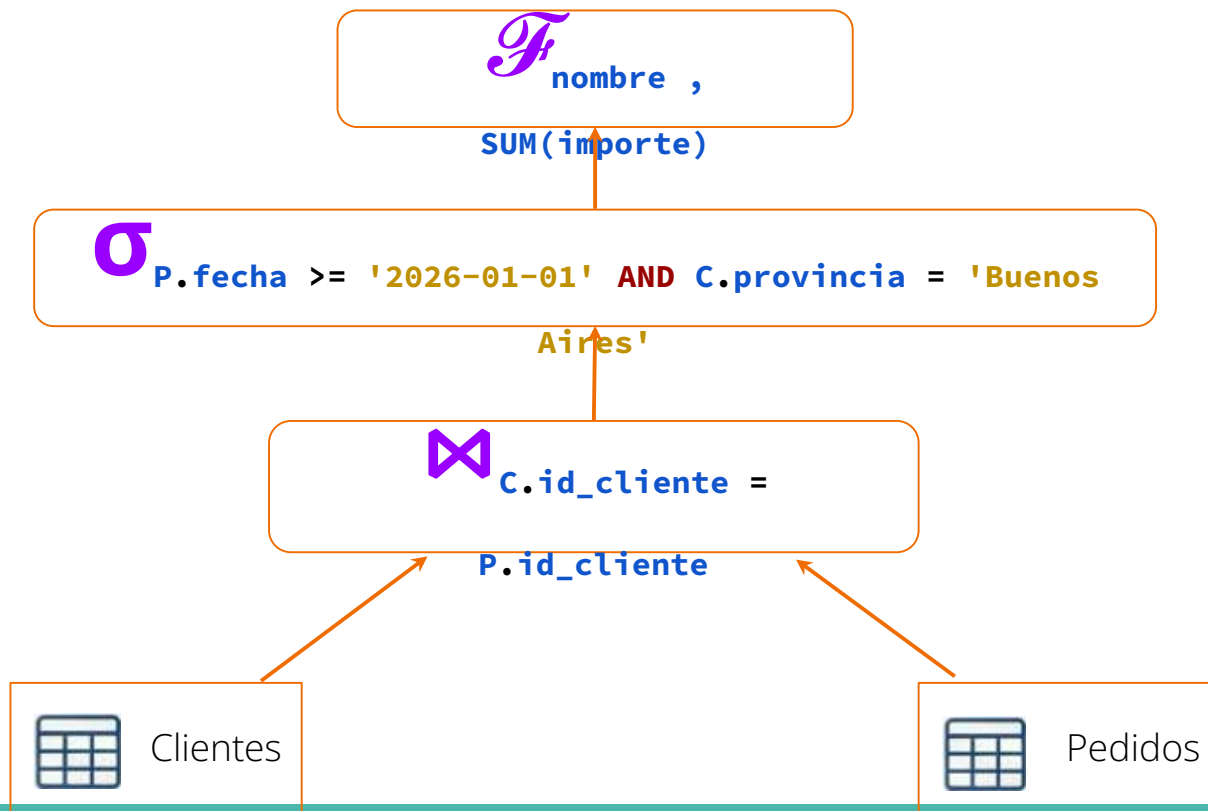
Árbol de la consulta de referencia - Nivel 3

Nivel 3: el nodo SELECCIÓN (filtro)



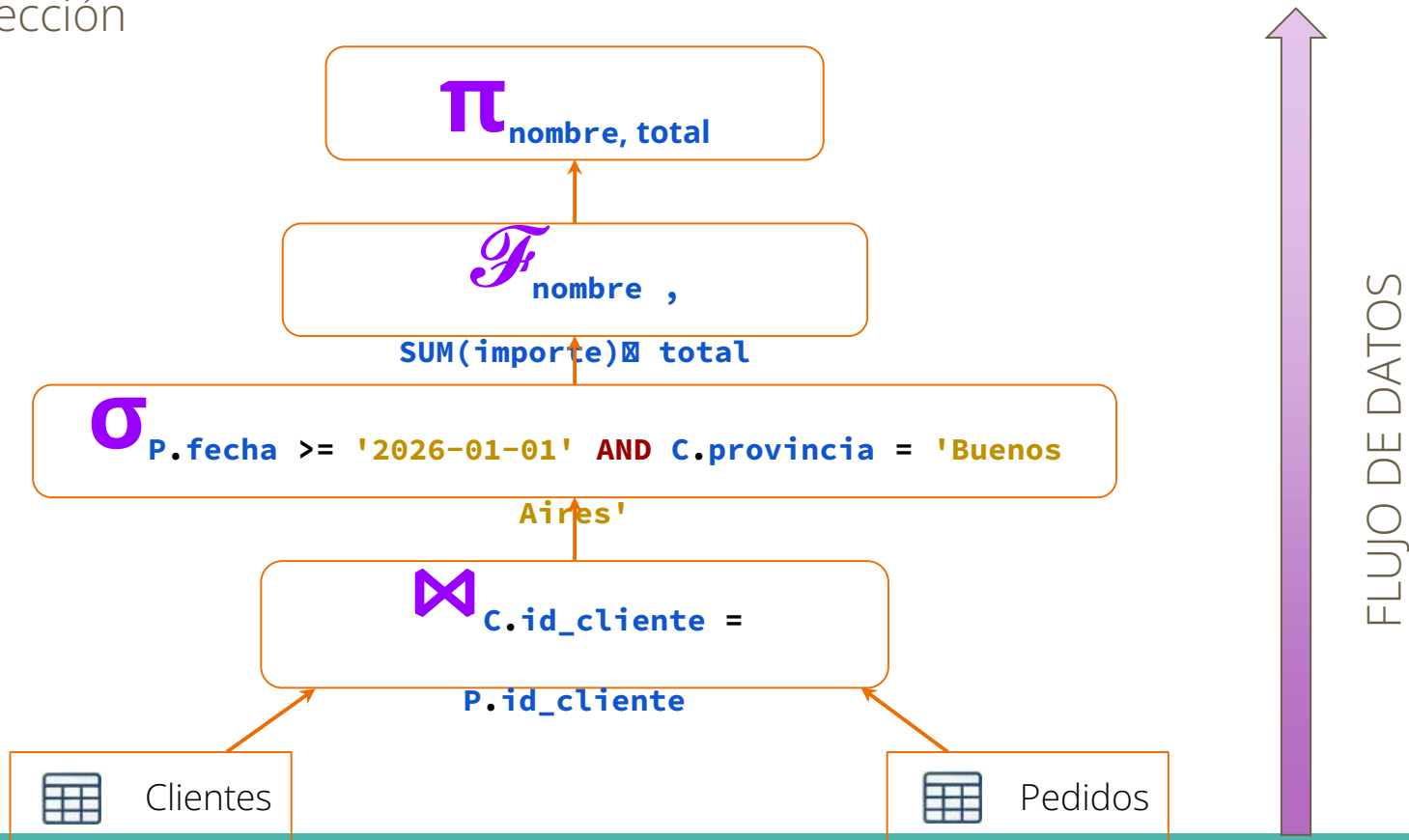
Árbol de la consulta de referencia - Nivel 4

Nivel 4: Agrupamiento y Agregación

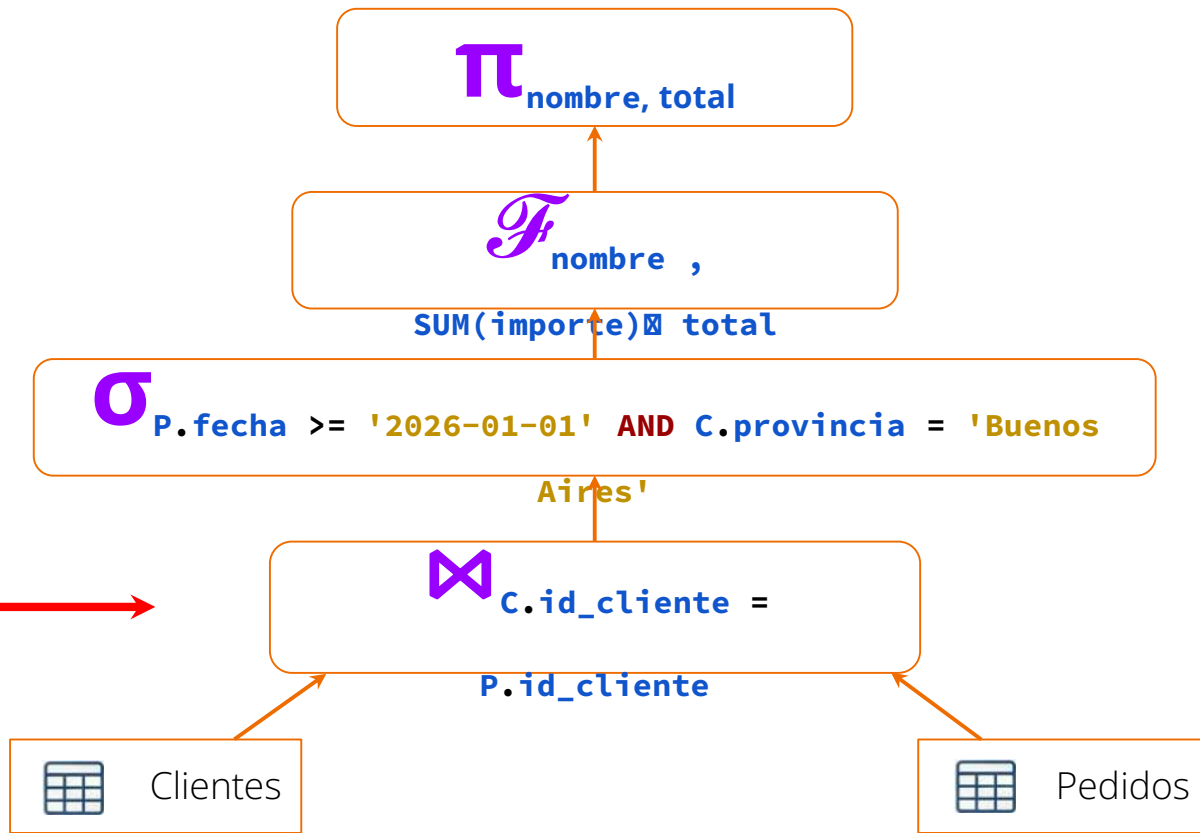


Árbol de la consulta de referencia - Nivel 5

Nivel 5: proyección



¿Es eficiente este árbol?



Problema: procesamos datos masivos que después descartamos

¿Por qué optimizar?

La optimización es esencial porque, para una misma consulta, existen cientos de formas de obtener el resultado, pero con costos abismalmente diferentes.

- El factor crítico: En sistemas de gran escala, el recurso más caro y lento es el Acceso a Disco (E/S). La velocidad de la CPU ha crecido mucho más rápido que la de los discos, por lo que el tiempo de ejecución suele estar dominado por la cantidad de bloques de datos que deben leerse o escribirse.
- Objetivo: El optimizador busca el plan que requiera el menor número de transferencias de bloques de disco

Consulta No Optimizada



Horas / Días

Consulta Optimizada



Segundos

Recursos Críticos

- CPU (Procesamiento)
- Memoria (Buffer Pool)
- I/O de Disco (El cuello de botella)

Fase 2: Optimización

El DBMS buscará encontrar la estrategia de ejecución más eficiente (menor tiempo de respuesta y uso de recursos).

- Optimización Lógica:
 - Se enfoca en la estructura de la consulta.
 - Utiliza equivalencias del Álgebra Relacional.
 - Transforma el árbol inicial en uno equivalente pero más eficiente (ej. mover filtros hacia abajo).
- Optimización Física:
 - Selecciona los algoritmos específicos para cada operación.
 - Ejemplo: ¿Para el JOIN usamos Nested Loops, Merge Join o Hash Join?
 - Decide el uso de puntos de acceso (¿Uso el índice o leo toda la tabla?).

Salida: El Plan de Ejecución. Es el conjunto de instrucciones finales que el motor de evaluación ejecutará físicamente.

Optimización heurística - Reglas lógicas

La heurística no usa estadísticas; usa reglas lógicas predefinidas que casi siempre funcionan.

- **Filtrar lo antes posible** (mover las selecciones hacia abajo): Reducir el número de filas antes de realizar operaciones costosas como el JOIN.
- Proyectar lo mínimo necesario (Proyecciones hacia abajo): Eliminar columnas que no se necesitan para reducir el tamaño de los datos en memoria.
- Reordenar Joins: Combinar primero las tablas que resulten en conjuntos de datos más pequeños.

Optimización heurística - Reglas lógicas

```
SELECT c.nombre, SUM(p.importe) AS total
FROM clientes C
JOIN pedidos P ON C.id_cliente = P.id_cliente
WHERE P.fecha >= '2026-01-01' AND C.provincia = 'Buenos Aires'
GROUP BY C.nombre;
```

¿Conviene filtrar pedidos antes del join?

Sí. Supongamos que *pedidos* tiene 1.000.000 de filas y solo 5.000 son de 2024, es mejor "unir" solo esas 5.000.

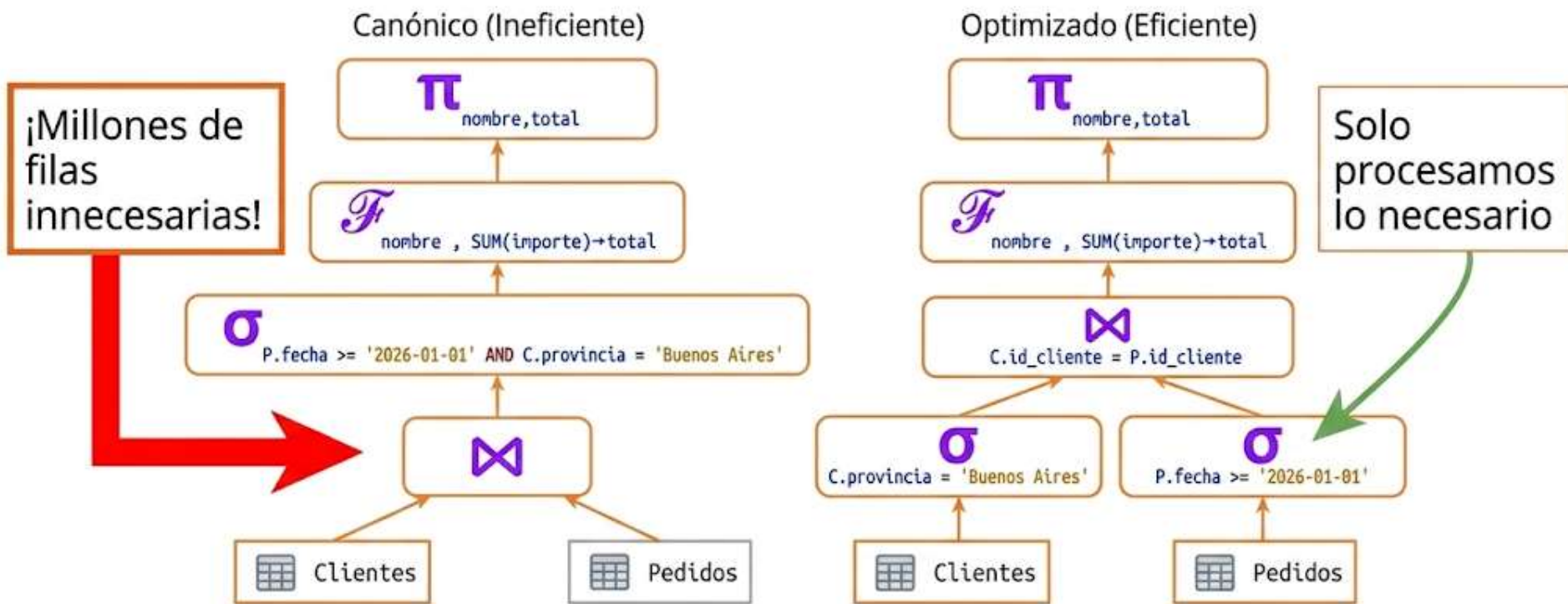
¿Conviene filtrar clientes antes del join?

Sí. Ídem al caso anterior.

¿Conviene **SELECT ***?

No. Si la tabla *clientes* tiene 50 columnas y solo usamos *nombre*, traer las otras 49 es desperdiciar memoria y ancho de banda.

Heurística: El arte de “bajar” los filtros



La regla de oro: Reducir el volumen de datos lo más pronto posible en el pipeline

Optimización Basada en Costos (CBO)

Cuando hay varios caminos lógicos correctos, el optimizador elige el que menos "cuesta" ejecutar en términos de hardware.

¿Qué información usa el optimizador? (Catálogo del DBMS)

- **Estadísticas:** El motor guarda metadatos sobre las tablas (se actualizan con comandos como ANALYZE).
- **Tamaño de las tablas:** Número de bloques o páginas que ocupa la tabla en el disco.
- **Cardinalidad:** El número total de filas y cuántos valores distintos hay en cada columna (selectividad).
- **Índices disponibles:** Evalúa si el costo de leer el índice + ir al disco es menor que leer toda la tabla (Full Table Scan).

Optimización Basada en Costos (CBO)

Fórmula Simplificada de Costo:

$$\text{Costo} \cong (N_{\text{bloques_leídos}} * W_{\text{IO}}) + (N_{\text{filas_procesadas}} * W_{\text{CPU}})$$

(Donde ***W*** representa el peso o importancia que el motor le da a cada recurso).

El optimizador elige el plan más barato, no el más "lindo" o elegante.

Ejemplo: Si la tabla clientes tiene solo 10 filas, el motor ignorará los índices y leerá toda la tabla porque es más barato que buscar en el árbol del índice.

¿Por qué el motor a veces ignora mis índices?

Cardinalidad: El "espectro" de tus datos

La **cardinalidad** es simplemente cuántos valores distintos hay en una columna en relación con el total de registros.

- **Alta Cardinalidad:** Valores casi únicos. Ejemplos: DNI, Email, Número de pedido.
- **Baja Cardinalidad:** Muchos valores repetidos. Ejemplos: Género (M/F/Otro), Estado de pedido (Pendiente/Enviado), Provincia.

Selectividad: El "filtro" de la consulta

La **selectividad** mide qué porcentaje de la tabla va a devolver tu consulta.

- **Alta Selectividad (Filtro fuerte):** Quieres el 0.1% de la tabla. El índice es **brillante** aquí porque te lleva directo al dato.
- **Baja Selectividad (Filtro débil):** Quieres el 60% de la tabla. Aquí es donde el motor empieza a dudar.

¿Por qué el motor a veces ignora mis índices?

El dilema del Motor: ¿Índice o Escaneo Total?

Imagina que tienes un libro de 1.000 páginas y un índice al final.

- **Caso A (Alta Selectividad):** Buscas un término que aparece solo en **1 página**. Miras el índice (1 segundo), vas a la página (1 segundo). Total: **2 segundos**.
- **Caso B (Baja Selectividad):** Buscas un término que aparece en **800 páginas**. Si usas el índice, tendrías que mirar el índice y saltar a una página, volver al índice y saltar a otra... ¡800 veces!
 - En este caso, al motor le sale más barato **leer el libro completo de corrido** (Full Table Scan) que estar saltando de adelante hacia atrás usando el índice.

¿Por qué? Porque la lectura secuencial (leer todo seguido) es físicamente más rápida para el hardware que la lectura aleatoria (saltar de un lado a otro).

¿Qué es el CBO y las Estadísticas?

El **CBO (Optimización Basada en Costos)** es el componente que toma la decisión final. Para decidir, consulta las **Estadísticas**: una "foto" que la base de datos guarda sobre tus tablas.

Si las estadísticas le dicen: *"Oye, en la columna 'Provincia', el 90% de los clientes son de 'Buenos Aires'"*, y tú haces un `WHERE provincia = 'Buenos Aires'`, el CBO dirá:

"No voy a usar el índice. Como casi todos son de Buenos Aires, me sale más barato leer la tabla entera de principio a fin que usar el índice para cada fila".

Resumen para recordar:

- **Índice** = Útil cuando buscas una aguja en un pajar.
- **Full Table Scan** = Útil cuando buscas el pajar entero.
- **El Motor es pragmático**: No usa el índice por "respeto" al programador, lo usa solo si realmente le ahorra tiempo de lectura en el disco.

El problema de “ayer andaba rápido y hoy anda lento”

En producción, el 90% de las veces que una consulta "se rompe" o se vuelve lenta de la noche a la mañana no es porque los datos crecieron, sino porque las estadísticas quedaron obsoletas y el Optimizador Basado en Costos (CBO) tomó una decisión errónea (por ejemplo, eligió hacer un Nested Loop en vez de un Hash Join porque creyó que había 10 filas cuando en realidad entraron 10.000 de golpe).

Postgres tiene un proceso automático llamado **Autovacuum** / **Autoanalyze** que corre de fondo actualizando esto, pero en cargas masivas de datos (ej. un proceso nocturno), el DBA Senior siempre ejecuta un ANALYZE manual al terminar de insertar para "despertar" al optimizador.

Planes de Ejecución

El Plan de Ejecución es la representación final y detallada de los pasos que el motor seguirá para resolver la consulta. Es donde la Optimización de Costos se vuelve realidad.

El plan nos dice:

- **Operadores Físicos:** ¿Usa un Index Seek (rápido, busca directo) o un Table Scan (lento, lee todo)?
- **Orden de las Operaciones:** Qué tablas se procesan primero.
- **Costo Relativo:** Qué parte de la consulta consume más recursos (ej. "Ese JOIN se lleva el 80% del tiempo").
- **Flujo de Datos:** El grosor de las flechas suele indicar cuántas filas viajan de un operador a otro.

Planes de Ejecución

El plan de ejecución es el script físico que el motor de evaluación va a seguir. Ya no hablamos de "qué" queremos, sino de "cómo" lo vamos a obtener.

¿Qué define un Plan?

- **Orden de operaciones:** Qué tablas se procesan primero y en qué secuencia se aplican los filtros y agregaciones.
- **Algoritmos:** La lógica específica para resolver cada operador (ej. cómo se cruzan los datos).
- **Métodos de acceso:** La estrategia para leer los datos desde el almacenamiento.

Planes de Ejecución

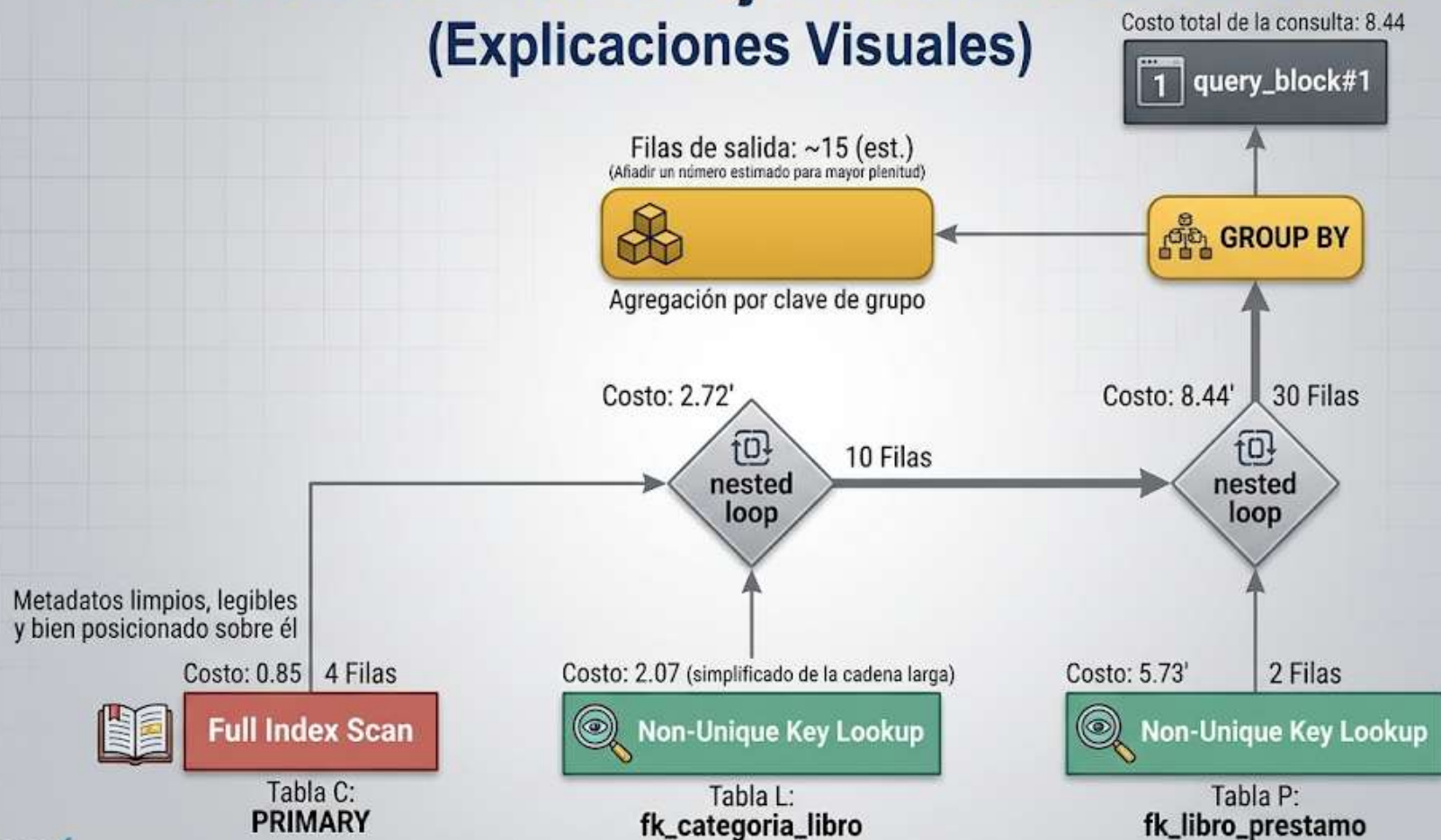
Ejemplos de operadores comunes

Tipo	Operador	Descripción
Acceso	Table Scan	Lee toda la tabla secuencialmente (Lento, se usa si no hay índices).
Acceso	Index Scan/Seek	Navega por la estructura del índice (Rápido, busca valores específicos).
Join	Nested Loop	Para cada fila de la Tabla A, busca en la Tabla B (Eficaz para conjuntos pequeños).
Join	Hash Join	Crea una tabla hash en memoria para cruzar datos (Ideal para grandes volúmenes).

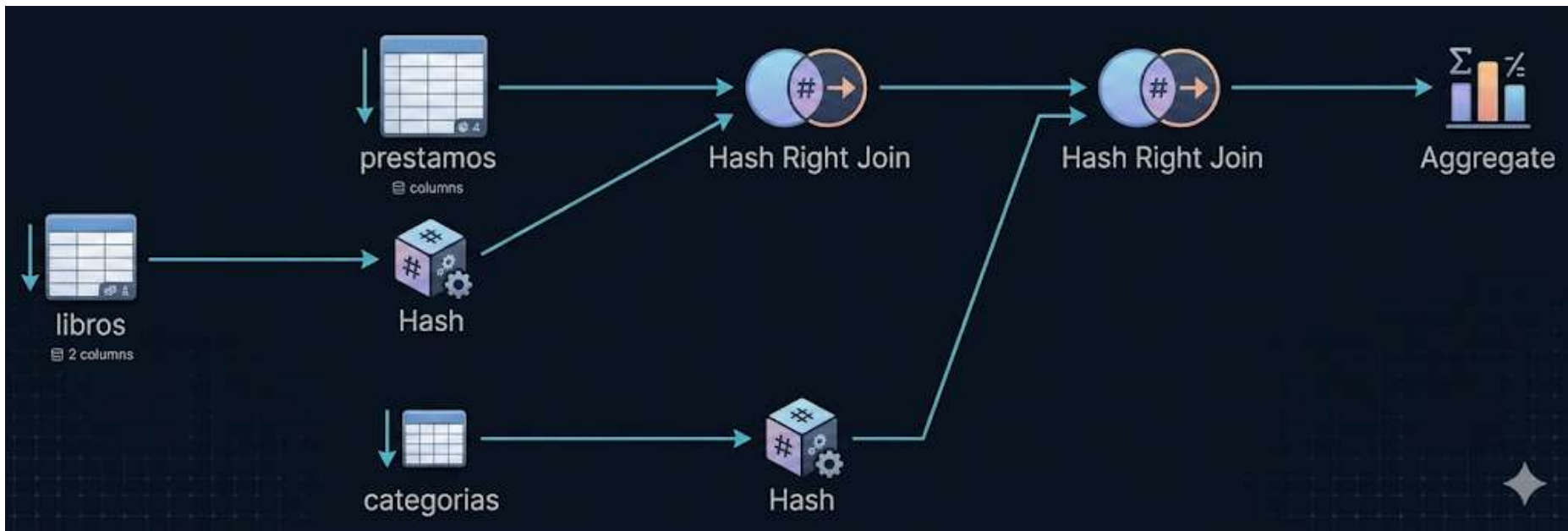
Planes de Ejecución

- Table Scan vs. Index Seek: Si hacemos analogía con un libro, un Table Scan es leer el libro de la página 1 a la 500 para encontrar una palabra. En cambio un Index Seek es ir al índice alfabético y saltar directo a la página correcta.
- El costo del algoritmo: el optimizador elige entre un Nested Loop o un Hash Join basándose en la cardinalidad (cantidad de filas) que calculó en la fase de costos. No hay un algoritmo "mejor" que otro, sino uno más adecuado para el volumen de datos.

Análisis del Plan de Ejecución de la Consulta (Explicaciones Visuales)



Planes de Ejecución - Ejemplos postgres



¿Cómo se ve un plan de ejecución?

QUERY PLAN

```
Sort (cost=717.34..717.59 rows=101 width=488) (actual time=7.761..7.774 rows=100 loops=1)
  Sort Key: t1.fivethous
  Sort Method: quicksort  Memory: 77kB
-> Hash Join (cost=230.47..713.98 rows=101 width=488) (actual time=0.711..7.427 rows=100 loops=1)
  Hash Cond: (t2.unique2 = t1.unique2)
    -> Seq Scan on tenk2 t2 (cost=0.00..445.00 rows=10000 width=244) (actual time=0.007..2.583 rows=10000 loops=1)
    -> Hash (cost=229.20..229.20 rows=101 width=244) (actual time=0.659..0.659 rows=100 loops=1)
      Buckets: 1024 Batches: 1 Memory Usage: 28kB
      -> Bitmap Heap Scan on tenk1 t1 (cost=5.07..229.20 rows=101 width=244) (actual time=0.080..0.526 rows=100 loops=1)
        Recheck Cond: (unique1 < 100)
        -> Bitmap Index Scan on tenk1_unique1 (cost=0.00..5.04 rows=101 width=0) (actual time=0.049..0.049 rows=100 loops=1)
          Index Cond: (unique1 < 100)

Planning time: 0.194 ms
Execution time: 8.008 ms
```

Índices

Un índice es una estructura de datos redundante (ocupa espacio extra) que el DBMS mantiene para encontrar registros rápidamente, evitando el Table Scan. Se clasifican según su función y su estructura física:

- Índice Primario (Primary Index):
 - Se crea automáticamente sobre la Primary Key (PK). Garantiza que no haya valores duplicados (Unicidad).
 - Suele ser el camino de búsqueda por defecto.
- Índice Agrupado (Clustered):
 - Estructura Física: Los datos de la tabla están ordenados físicamente en el disco según este índice.
 - Importante: Solo puede haber uno (no se pueden ordenar físicamente los datos por mas de una columna).
 - Nota: En muchos motores, la PK es automáticamente el Clustered Index.
- Índice Secundario (Secondary / Non-Clustered):
 - Es una estructura "extra" que apunta a la ubicación real de los datos.
 - No altera el orden físico de la tabla.
 - Podemos tener múltiples índices secundarios (ej: uno por fecha, otro por email).
- Índice Multicolumna (Composite Index):
 - Índice compuesto por dos o más campos (ej: apellido + nombre).
 - Clave: El orden de las columnas importa (Regla del "prefijo más a la izquierda").
 - Útil para consultas que filtran por varios criterios simultáneamente.

Índices

Los índices aceleran los **SELECT**, pero ralentizan los **INSERT**, **UPDATE** y **DELETE**, ya que el motor debe actualizar el índice cada vez que los datos cambian.

Sintaxis:

```
CREATE INDEX idx_nombre_indice ON tabla(columna);
```

Para nuestra consulta de referencia ¿qué índices nos convendría tener?

EL DILEMA DEL INDEXADO: VELOCIDAD VS. MANTENIMIENTO

BENEFICIO



LECTURAS (SELECT)
hasta **1000x**
más rápidas.

COSTO



Cada índice nuevo
**ralentiza INSERT,
UPDATE, DELETE.**



Alta Velocidad de Lectura
(SELECT)



CONSULTAS DE REPORTE

Muchos Índices

Alta Velocidad de Escritura
(INSERT, UPDATE, DELETE)



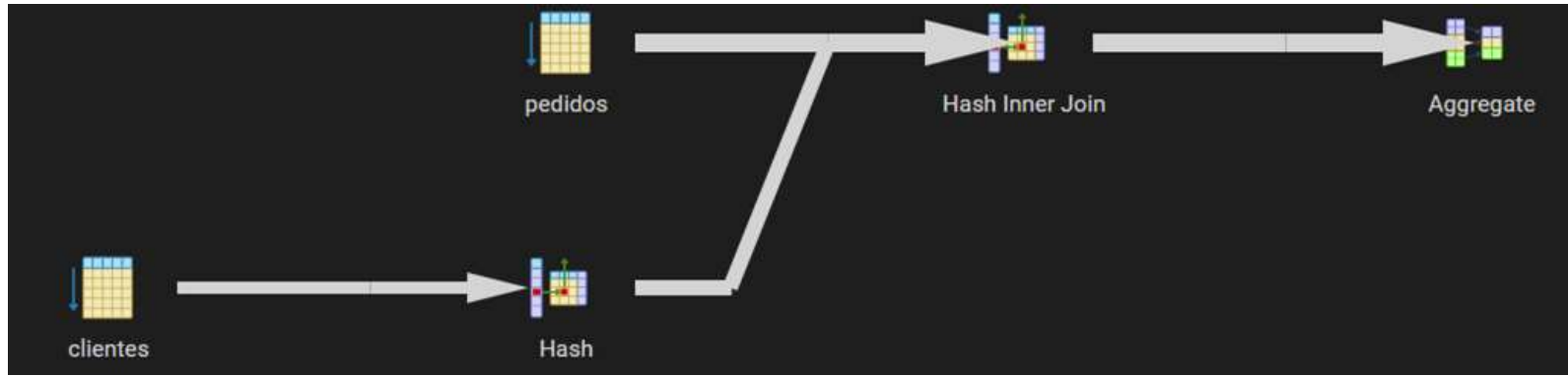
TRANSACCIONES DE VENTA

Pocos Índices



CONSEJO SENIOR:
"Indexá con estrategia,
no por miedo".

Ejecución de la consulta sin índices



Ejecución de la consulta sin índices

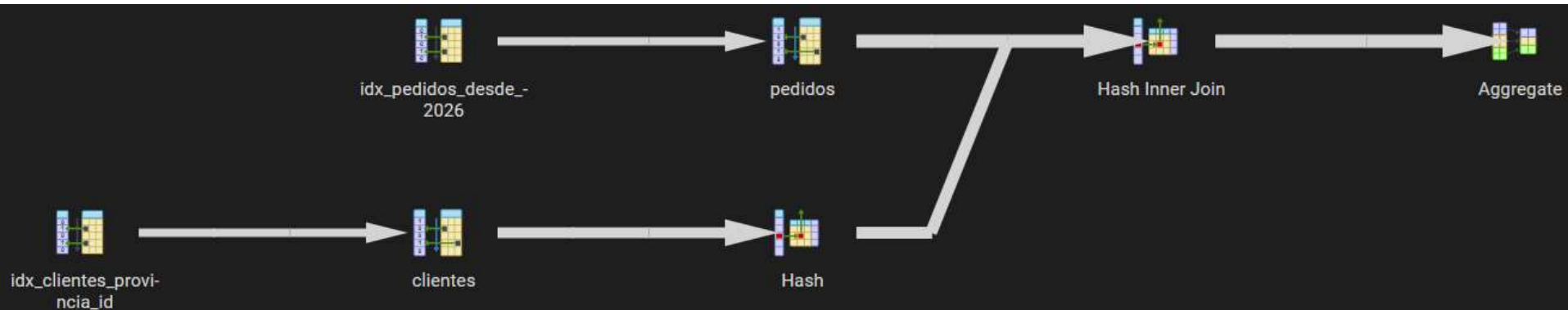
Node	Timings	
	Exclusive	Inclusive
→ Aggregate (cost=45015.17..45105.71 rows=7243 width=53) (actual=171.124..172.... Buckets: Batches: Memory Usage: 2449 kB	7.914 ms	172.713 ms
→ Hash Inner Join (cost=1363.46..44960.85 rows=7243 width=29) (actual=6.581... Hash Cond: (p.id_cliente = c.id_cliente)	8.368 ms	164.799 ms
→ Seq Scan on pedidos as p (cost=0..43534 rows=24148 width=24) (actual... Filter: (fecha >= '2026-01-01'::date) Rows Removed by Filter: 1975642	149.969 ms	149.969 ms
→ Hash (cost=1176..1176 rows=14997 width=21) (actual=6.463..6.463 row... Buckets: 16384 Batches: 1 Memory Usage: 953 kB	1.725 ms	6.463 ms
→ Seq Scan on clientes as c (cost=0..1176 rows=14997 width=21) (act... Filter: ((provincia)::text = 'Buenos Aires'::text) Rows Removed by Filter: 34928	4.739 ms	4.739 ms



Ejecución de la consulta con índices

```
CREATE INDEX idx_clientes_provincia_id ON clientes (provincia, id_cliente);  
CREATE INDEX idx_pedidos_fecha_cliente ON pedidos (fecha, id_cliente);  
CREATE INDEX idx_pedidos_cliente ON pedidos (id_cliente);  
CREATE INDEX idx_pedidos_desde_2026 ON pedidos(fecha) WHERE fecha >= '2026-01-01';
```

```
ANALYZE clientes;  
ANALYZE articulos;  
ANALYZE pedidos;
```



Ejecución de la consulta con índices

Node	Timings	
	Exclusive	Inclusive
→ Aggregate (cost=21499.99..21591 rows=7281 width=53) (actual=39.323..41.271 row... Buckets: Batches: Memory Usage: 2449 kB	6.378 ms	41.271 ms
→ Hash Inner Join (cost=1555.48..21445.38 rows=7281 width=29) (actual=11.332.... Hash Cond: (p.id_cliente = c.id_cliente)	7.061 ms	34.893 ms
→ Bitmap Heap Scan on pedidos as p (cost=221.31..20047.96 rows=24098 wi... Recheck Cond: (fecha >= '2026-01-01'::date) Heap Blocks: exact=13575	17.961 ms	20.57 ms
→ Bitmap Index Scan using idx_pedidos_desde_2026 (cost=0..215.29 row...	2.609 ms	2.609 ms
→ Hash (cost=1145.33..1145.33 rows=15107 width=21) (actual=7.25..7.263 r... Buckets: 16384 Batches: 1 Memory Usage: 953 kB	2.253 ms	7.263 ms
→ Bitmap Heap Scan on clientes as c (cost=405.49..1145.33 rows=15107... Recheck Cond: ((provincia)::text = 'Buenos Aires'::text) Heap Blocks: exact=551	4.009 ms	5.01 ms
→ Bitmap Index Scan using idx_clientes_provincia_id (cost=0..401.7... Index Cond: ((provincia)::text = 'Buenos Aires'::text)	1.001 ms	1.001 ms



Índices Compuestos (Los "Combos")

```
CREATE INDEX idx_clientes_provincia_id ON clientes (provincia, id_cliente);
```

```
CREATE INDEX idx_pedidos_fecha_cliente ON pedidos (fecha, id_cliente);
```

Por qué: Estos no son índices simples, son **índices compuestos**.

La magia: Al poner `provincia` primero en el de clientes, el motor filtra rapidísimo por "Buenos Aires". Pero como también incluiste el `id_cliente` en el mismo índice, el motor **no necesita ir a la tabla principal** para buscar el ID para el JOIN. Ya lo tiene todo en el índice. A esto se le llama *Index-Only Scan*.

Orden: Siempre se pone primero la columna que usas en el `WHERE` (filtro) y después la que usas en el `JOIN`.

Índice en la Foreign Key (FK)

```
CREATE INDEX idx_pedidos_cliente ON pedidos (id_cliente);
```

Por qué: Es una regla de oro. Las claves foráneas (`id_cliente` en la tabla `pedidos`) siempre deben estar indexadas.

La razón: Cada vez que haces un `JOIN`, el motor tiene que buscar coincidencias. Si no hay índice aquí, por cada cliente de Buenos Aires, el motor tendría que leer la tabla `pedidos` entera para ver cuáles le pertenecen. El índice convierte esa búsqueda en algo instantáneo.

Índice Parcial (El ahorro de recursos)

```
CREATE INDEX idx_pedidos_desde_2026 ON pedidos(fecha) WHERE fecha >= '2026-01-01';
```

Por qué: Esto es nivel **Senior**. En lugar de indexar todas las fechas desde que se creó la empresa (capaz hace 10 años), solo indexas lo que te interesa.

Ventajas: * **Menos espacio:** El índice es mucho más pequeño.

- **Más rápido:** Al ser pequeño, cabe entero en la memoria RAM (Buffer Cache).
- **Mantenimiento:** Cuando insertas pedidos de 2024, este índice no se actualiza, ahorrando tiempo de escritura.

El comando ANALYZE (Actualizar el "Cerebro")

ANALYZE clientes;
ANALYZE articulos;
ANALYZE pedidos;

Por qué: Como vimos en la transparencia del **CBO (Optimizador Basado en Costos)**, el motor toma decisiones basadas en estadísticas.

Qué hace: ANALYZE no crea índices, sino que le cuenta al motor: "Mira, en la tabla clientes ahora tenemos un 30% de gente de Buenos Aires y un total de 1 millón de filas".

Sin esto: Si la base de datos cree que la tabla tiene 10 filas (porque así era cuando se creó) pero ahora tiene un millón, el optimizador elegirá una "ruta" (plan de ejecución) pésima que hará que la consulta tarde minutos en lugar de milisegundos.

NOTA IMPORTANTE sobre los índices

Recién creamos 4 índices para mostrar cómo el motor los combina, pero en la vida real un Senior intentaría crear un solo índice compuesto estratégico que cubra la consulta, o indexaría sólo la columna más selectiva (ej. fecha), para no castigar las escrituras.

Esta es la diferencia entre **la teoría académica** (donde mostramos todas las herramientas posibles) y **la ingeniería de producción** (donde buscamos el equilibrio entre velocidad y costo).

Esa frase dice eso por tres razones fundamentales que separan a un "programador que sabe SQL" de un "especialista en bases de datos"

Si tienes 4 índices, el motor tiene que hacer 5 escrituras físicas en lugar de una. En un sistema con miles de ventas por minuto, tener muchos índices puede hacer que la aplicación se "congele" o se ponga lenta porque el disco está ocupado reorganizando los índices.

El Principio de Selectividad (La columna "Ganadora")

A veces, un solo índice es tan potente que los demás sobran.

Imagina que tienes 1.000.000 de pedidos:

- Filtrar por `provincia = 'Buenos Aires'` te deja con **400.000** filas (No reduce tanto).
- Filtrar por `fecha >= '2026-01-01'` (suponiendo que es hoy) te deja con solo **500** filas.

Si el índice de `fecha` ya te redujo el pajar de un millón a solo 500 pajas, buscar el cliente y la provincia en esas 500 filas restantes es **insignificante** para el procesador. En este caso, el Senior diría: *"Borremos los otros 3 índices, con el de fecha me basta y me sobra, y así no ralentizo las ventas".*

Único Índice Estratégico

se basa en crear un **Índice Compuesto** diseñado "a medida" para una consulta específica. En lugar de darle al motor varias herramientas pequeñas (índices simples), le das una sola "superherramienta" que contiene todo lo que necesita.

Para nuestra consulta de referencia, el índice estratégico sería este:

```
CREATE INDEX idx_estratégico_pedidos  
ON pedidos (fecha, id_cliente, importe);
```

El Orden de las Columnas (La Regla de Oro)

En un índice compuesto, el orden **SÍ importa**. Se lee de izquierda a derecha.

- **Primero, el Filtro (*fecha*):** El motor va directo a la sección del índice donde empiezan los datos de 2026. Al estar indexado, no "busca", sino que "salta" al lugar correcto.
- **Segundo, el Join (*id_cliente*):** Una vez que el motor ya tiene las filas de 2026, el *id_cliente* está ahí mismo, pegado a la fecha. No tiene que ir a la tabla principal a buscar quién es el cliente.

El "Covering Index" (Índice de Cobertura)

Fíjate que incluimos `importe` al final del índice.

¿Por qué, si no filtramos por importe?

Porque nuestra consulta pide `SUM(p.importe)`. Si el importe está en el índice, el motor puede calcular la suma **sin tocar nunca la tabla original**.

Es como si quisieras saber el precio total de unos libros:

- **Sin cobertura:** Miras el catálogo (índice), anotás el número de estante, vas al estante, sacás el libro y mirás el precio en la tapa.
- **Con cobertura:** El precio ya está anotado en el catálogo. No necesitas caminar hasta el estante.

A esto en performance se le llama **Index-Only Scan**. Es el "santo grial" de la velocidad en SQL.

Desafío: ¿Cuál es el error de performance?



Caso (Desarrollador Junior)

```
SELECT nombre FROM usuarios  
WHERE YEAR(fecha_alta) = 2024;
```



Pregunta a la clase

¿Por qué esta consulta es lenta aunque exista un **índice** en **fecha_alta**?



Respuesta

Al usar una función (YEAR), el motor no puede usar el índice directamente (SARGability).



Versión Senior (Solución Correcta)

```
WHERE fecha_alta >= '2024-01-01' AND fecha_alta <= '2024-12-31';
```

Vistas

Una vista es una "**tabla virtual**". No almacena los datos físicamente (en su forma estándar), sino que almacena una consulta **SELECT** que se ejecuta cada vez que la vista es invocada.

Ventajas:

- **Simplificación:** Ocultan la complejidad de consultas con múltiples JOINS o cálculos complejos. El programador solo hace **SELECT * FROM vista**.
- **Seguridad:** Permiten mostrar a un usuario sólo ciertas columnas o filas de una tabla, protegiendo datos sensibles (ej: sueldos o contraseñas).
- **Independencia Lógica:** Si la estructura de las tablas cambia, solo se modifica la vista y la aplicación sigue funcionando igual.

Vistas - tipos y mantenimiento

- Vistas Estándar: Se recalculan en cada uso. No ocupan espacio en disco, pero no ahorran tiempo de ejecución.
- Vistas Actualizables: Bajo ciertas condiciones (usualmente si no tienen GROUP BY o DISTINCT), permiten hacer INSERT o UPDATE sobre ellas.
- Vistas Materializadas: El resultado se guarda físicamente en el disco.
 - Ventaja: Respuesta instantánea para reportes pesados.
 - Desafío: Deben "refrescarse" cuando los datos base cambian (Costo de mantenimiento).

Por lo tanto, no todas las vistas mejoran la performance.

Vistas - ejemplo

Creamos la vista para el departamento de ventas de Buenos Aires:

```
CREATE VIEW reporte_ventas_bsas_2026 AS  
  
SELECT c.nombre, SUM(p.importe) AS total, COUNT(p.id_pedido) AS  
cant_pedidos  
  
FROM clientes C JOIN pedidos P ON C.id_cliente = P.id_cliente  
  
WHERE P.fecha >= '2026-01-01' AND C.provincia = 'Buenos Aires'  
  
GROUP BY C.nombre;
```

La podemos usar para recuperar los datos:

```
SELECT * FROM reporte_ventas_bsas_2026 ;
```

Vistas materializadas - sintaxis

Sintaxis:

```
CREATE MATERIALIZED VIEW nombre_de_la_vista AS  
SELECT ...  
WITH [NO] DATA;
```

- **WITH DATA:** (Por defecto) Genera la vista y la llena de datos inmediatamente.
- **WITH NO DATA:** Crea la estructura de la vista, pero la deja vacía hasta que decidas llenarla.

*Nota: en SQL Server el concepto es el mismo pero la sintaxis cambia. No existe **CREATE MATERIALIZED VIEW**; Lo que se hace es crear una vista normal y luego crearle un índice agrupado único. Eso "materializa" los datos en el disco.*

Vistas materializadas - ciclo de vida

Una vista materializada es una "foto" del pasado. Si los datos de las tablas base cambian, la vista materializada NO se actualiza sola.

Para actualizarla, la sintaxis es:

REFRESH MATERIALIZED VIEW nombre_de_la_vista;

Vistas materializadas - resumen

Nota de color: Silberschatz menciona que la elección de qué vistas materializar y cuándo refrescarlas es un problema de optimización en sí mismo llamado "*View-Selection Problem*"

Característica	Vista Estándar	Vista Materializada
Almacenamiento	Solo guarda la consulta (metadatos).	Guarda los datos reales en disco.
Rendimiento	El costo es igual a ejecutar la consulta.	El acceso es instantáneo ($O(1)$ o $O(\log n)$ si tiene índices).
Frescura	Siempre está al día (tiempo real).	Se vuelve obsoleta hasta que se refresca.
Uso Ideal	Seguridad y simplicidad.	Reportes pesados y Data Warehousing.

Vistas materializadas - ejemplo

```
CREATE MATERIALIZED VIEW reporte_ventas_bsas_2026 AS  
SELECT c.nombre, SUM(p.importe) AS total, COUNT(p.id_pedido) AS  
cant_pedidos  
FROM clientes C JOIN pedidos P ON C.id_cliente = P.id_cliente  
WHERE P.fecha >= '2026-01-01' AND C.provincia = 'Buenos Aires'  
GROUP BY C.nombre  
WITH NO DATA;  
  
REFRESH MATERIALIZED VIEW reporte_ventas_bsas_2026 ;  
  
SELECT * FROM reporte_ventas_bsas_2026 ;  
  
DROP MATERIALIZED VIEW reporte_ventas_bsas_2026 ;
```

Resumen

Para conectar todo lo que vimos hoy:

- Declaración (SQL): El programador dice qué quiere.
- Análisis: El motor genera el Árbol de Consulta (Álgebra Relacional).
- Optimización Lógica (Heurística): Se aplican reglas ("Bajar" filtros y proyecciones).
- Optimización Física (Costos): El motor consulta las estadísticas y decide: ¿Uso un índice o leo toda la tabla? ¿Qué algoritmo de Join es más barato?
- Plan de Ejecución: Se genera la hoja de ruta final.
- Evaluación: El motor de ejecución accede a los Índices (B-Trees) para evitar leer el disco innecesariamente y entrega el resultado (que puede estar encapsulado en una Vista).

Cómo aplicar esto desde hoy

- No confíes a ciegas en las consultas que pensaste: usá siempre **EXPLAIN ANALYZE** o similar para validar tus consultas pesadas.
- El problema del **SELECT ***: Traer columnas de más anula optimizaciones de índices y satura la memoria.
- Índices con criterio:
 - Indexá siempre las FK (Foreign Keys) para acelerar los Joins.
 - Cuidado con el exceso: cada índice nuevo ralentiza tus INSERT y UPDATE.
- Aprovechá la Heurística: Ayudá al motor escribiendo filtros claros y evitando funciones sobre columnas indexadas (ej: **WHERE YEAR(fecha) = 2024** suele anular el índice; es mejor **WHERE fecha >= '2024-01-01'**).
- Vistas para el orden: Usalas para simplificar la vida de tu equipo y proteger datos, pero no esperes que mejoren la velocidad si no son materializadas.

Conclusión y aplicación en la carrera profesional

"Una base de datos no es lenta por el volumen de sus datos, sino por la ineficiencia de nuestras peticiones."

La diferencia entre un Senior y un Junior es entender el Plan de Ejecución.

- Perfil Junior: Se enfoca en la funcionalidad. Su éxito es que la consulta devuelva el dato correcto (el qué). Suele ver a la base de datos como una "caja negra" mágica.
- Perfil Senior: Se enfoca en la eficiencia. Su éxito es que la consulta sea escalable (el cómo).
 - Domina el Plan de Ejecución.
 - Diseña índices basados en patrones de acceso real.
 - Entiende que cada JOIN o DISTINCT tiene un costo en hardware y un impacto en el negocio (latencia = pérdida de usuarios).

El objetivo de esta clase es darles las herramientas para que dejen de ver la base de datos como una caja negra y empiecen a verla como un motor de alta precisión.

Tu Checklist para el mundo real



[] ¿Revisé el Plan de Ejecución con EXPLAIN?



[] ¿Evité el SELECT * para no saturar memoria?



[] ¿Mis filtros son 'limpios' (sin funciones sobre columnas)?



[] ¿Están las estadísticas actualizadas (ANALYZE)?



[] ¿Es esta consulta candidata para una Vista Materializada?

“No escribas consultas que funcionen, escribe consultas que escalen”.

**No te hice un SELECT *.
Te filtré con intención, te
ordené por prioridad y supe
que no eras consulta
temporal. Eras la tabla donde
quería quedarme a vivir.**